

Hardware and Architecture

1. Describe the general functions of the PC BIOS and how its embedded routines are used by your boot loader.
2. What is the significance of the 1024 hard disk cylinder to Linux boot loaders?
3. Name three files in the /proc filesystem that contain information on system resource allocations.
4. Which of the following SCSI interfaces has the fastest data transfer rates: SCSI-1, SCSI-2, Ultra SCSI, or Fast-Wide SCSI?
5. What command is used to obtain serial port information on a Linux system?
6. What driver is used for USB hard drives?

Exercises

1. Boot your PC and enter the BIOS configuration utility. Review settings for correct hardware date and time.
2. Examine the enabled serial and parallel ports. Can you manually configure the interrupts and I/O ports assigned to them?
3. Review the options available for system boot. Does the system support booting from a CD-ROM? Change the default boot order.
4. Examine your modem and sound external interfaces on your PC. Are the devices built into your motherboard or independent expansion cards? If it is a card, identify if it is ISA or PCI.
5. Determine if your installed modem is a software modem.
6. If you have a SCSI controller, reboot your PC and enter the SCSI BIOS. What device number is selected, if any, for boot? How are the controller's onboard terminators configured? What data rate is the controller configured for?
7. Examine the kernel's interrupt assignments by executing `cat /proc/interrupts`. Are your devices reported correctly? Are there any surprises?
8. Review output from `cat /proc/dma` and `cat /proc/ioports` (discussed in the section "Using the /proc filesystem" in Chapter 3)
9. Create a list of all installed PCI devices using `lspci`. Note the devices built into your motherboard.
10. Use `minicom` to attach to your modem. For example:

```
# minicom /dev/modem
Welcome to minicom 1.82
OPTIONS: History Buffer, F-key Macros,
Search History Buffer, I18n
Compiled on Mar 21 1999, 21:10:56.
Press CTRL-A Z for help on special keys
AT S7=45 S0=0 L1 V1 X4 &c1 E1 Q0
OK
AT
OK
```

Does the modem respond to the AT command with OK? try manually dialing your ISP and watch the output of the modem.

11. Connect a USB device (mouse, printer, etc.) to your system. Run `lsmod` to verify that the appropriate driver loaded.

Linux Installation and Package Management

1. Why is the `/var` directory usually located in a partition of its own?
2. As a system administrator for a network with many workstations and a central NFS file server, how can you safely share `/usr` with your users while still maintaining control of its contents?
3. Describe how to create a tar archive and how its contents are extracted.
4. In general terms, describe the procedure used to compile and install free or open source software from source code.
5. What is a shared library? How can you determine what library dependencies exist in a compiled executable?
6. Briefly describe the major functional modes of `rpm`.
7. Why might a Debian Linux administrator use `dpkg -i` instead of simply `dpkg -i` to install a package?

Exercises

1. In a shell, examine your disk layout using `cfdisk` or `fdisk`. For example:

```
# fdisk
Command (m for help): p
Disk /dev/sda: 255 heads, 63 sectors, 1109 cylinders
Units = cylinders of 16065 * 512 bytes
   Device Boot      Start         End      Blocks   Id  System
/dev/sda1             1           51     409626    83  Linux
/dev/sda2             52        1109    8498385     5  Extended
/dev/sda5             52           90     313236    83  Linux
/dev/sda6             91           97      56196    83  Linux
/dev/sda7             98          136     313236    83  Linux
/dev/sda8            137          264    1028128+    83  Linux
/dev/sda9            265          519    2048256    83  Linux
/dev/sda10           520          532     104391    83  Linux
/dev/sda11           533          545     104391    82  Linux swap
/dev/sda12           546        1109    4530298+    83  Linux
```

Is the entire disk consumed by the existing filesystems?

2. Examine how system directories are mapped to disk partitions on your system. Are `/var` and `/tmp` in their own partitions? Is `/boot` in its own partition within cylinder 1024? Is the root filesystem relatively small?
3. Where is LILO installed on your system? If it is installed in the boot sector, does your configuration allow for multiple boot scenarios? If it is installed in the root partition, is it within the first 1024 cylinders?

4. Locate a tarball (from freshmeat.net, for example), and install it on your system with the following steps:

1. Unpack it using `tar xzvf file`.
 2. Configure with `./configure`.
 3. Build the software using `make` as directed in the documentation.
 4. Install the software using the instructions provided.
- Were there any difficulties with this procedure?

5. Use `ldd` to examine library dependencies of executable programs on your system. For example:

```
# ldd 'which xterm'
libXaw.so.7 => /usr/X11R6/lib/libXaw.so.7 (0x40019000)
libXmu.so.6 => /usr/X11R6/lib/libXmu.so.6 (0x4006a000)
libXt.so.6 => /usr/X11R6/lib/libXt.so.6 (0x4007e000)
libSM.so.6 => /usr/X11R6/lib/libSM.so.6 (0x400c7000)
libICE.so.6 => /usr/X11R6/lib/libICE.so.6 (0x400d0000)
libXpm.so.4 => /usr/X11R6/lib/libXpm.so.4 (0x400e6000)
libXext.so.6 => /usr/X11R6/lib/libXext.so.6 (0x400f4000)
libX11.so.6 => /usr/X11R6/lib/libX11.so.6 (0x40101000)
libncurses.so.4 => /usr/lib/libncurses.so.4 (0x401c4000)
libc.so.6 => /lib/libc.so.6 (0x40201000)
/lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
```

6. Using a system that utilizes `dpkg`, obtain a list of all packages installed under `dpkg` management with `dpkg -l | less`. Find a package in the list that looks unfamiliar, and query information about the package using `dpkg -s pkg_name`.

7. Using a system that utilizes `RPM`, obtain a list of all packages installed under `RPM` management with `rpm -qa | less`. Find a package in the list that looks unfamiliar, and query information about the package using `rpm -qi pkg_name`.

GNU and Unix Commands

1. Describe the difference between shell variables and environment variables.
2. Compare and contrast built-in and explicitly defined commands and those found in `PATH`.
3. After a lengthy session of file manipulation on the command line, what will `!ls` produce?
4. What program was the source for the default history editing key bindings in `bash`?
5. Explain the notion of pipes as they refer to shell capabilities, and illustrate using an example of two or more filter programs.
6. Explain the `-p` option to `cp` and give an example of why it is necessary.
7. Give two examples of files matched by the wildcard `??[!1-5]`.
8. Name the three standard I/O streams and their functions.
9. Give an example of the redirection operator, `>`, and describe how the outcome would be different using the `>>` operator.
10. What process is the ultimate ancestor of all system processes? Give both the PID and the program name.

11. Name three common utilities used for process monitoring.
12. What happens to a typical daemon when it receives SIGHUP? How would the behavior be different if it received SIGKILL?
13. Compare and contrast background and foreground jobs, and state the syntax to put a command in the background on the command line.
14. Explain the relationship between a process's nice number and its execution priority.
15. What two classifications of characters make up regular expressions?
16. How are the regular expressions `[A-Z]*` and `^[A-Z]*$` different?
17. What is the difference between executing `:q` versus `:q!` in vi?
18. What does it mean to put vi into command mode?

Exercises

1. Start a bash shell in a console or terminal window and enter the following commands:

```
$ MYVAR1="Happy"
$ MYVAR2="Birthday"
$ export MYVAR1
$ bash
$ echo $MYVAR1 $MYVAR2
$ exit
$ echo $MYVAR1 $MYVAR2
```

1. Was the behavior of the two echo commands identical?
2. If so, why? If not, why not?
3. What happened immediately after the bash command?
4. Which variable is an environment variable?

2. Continuing the previous exercise, enter Ctrl-P until you see the last echo command. Enter Ctrl-P again.

1. What do you see?
2. Why wasn't it the exit command?
3. Enter Ctrl-P again so that the export command is displayed. Add a space and MYVAR2 so that the line now looks like this:

```
$ export MYVAR1 MYVAR2
```

What happens when you enter this command?

3. Still continuing the previous exercise, enter the command `!echo`. Does anything change as a result of the revised export command?
4. The `file` command is used to examine a file's contents and displays the file type. Explain the result of using `file` as follows:

```
$ cd / ; file $(ls | head -10)
```

5. Execute this command on your system:

```
$ cut -d: -f1 /etc/passwd | fmt -w 20 | head -1
```

1. What was displayed?
2. How many lines of output did you see? Why?
3. What was the width of the output? Why?

6. Execute the following sed substitution command and explain why it might be used on /etc/passwd:

```
$ sed 's/:[^:]*:/:---:/' /etc/passwd | less
```

7. Execute this command:

```
$ cd /sbin ; ls -li e2fsck fsck.ext2
```

1. What is the significance of the first field of the output?
2. Why is it identical for both listings?
3. Why are the file sizes identical?

8. Execute the following command sequence and explain the result at each step (this example assumes that cp is not aliased to cp -i, which is a common default alias):

```
$ cd  
$ cp /etc/skel .  
$ cp -r /etc/skel .  
$ cp -rfv /etc/skel .  
$ cp -rfvp /etc/skel .
```

9. Remove the directory created in the previous exercise, using rmdir and/or rm. Which command can complete the task in a single step?

10. Explain when the wildcard {htm,html} might be useful.

11. Give an example of how the wildcard *. [Tt][Xx][Tt] could be used with directory listings.

12. What can be said about filenames matched by the *.* wildcard?

13. Experiment with redirecting the output of ls as follows:

```
$ cp /etc/skel . 2> info.txt
```

1. How is the terminal output different than that observed in Exercise 8?
2. What is written to info.txt ?

14. Experiment with ps, pstree, and top to monitor active processes on your system. Include top's interactive commands.

15.If you have Apache running, use ps (and perhaps grep) to identify the httpd process and its PID, which is owned by root. Send that process the HUP signal as follows:

```
$ kill -SIGHUP pid
```

Using tail, examine the Apache error log (the location of your log file may differ):

```
$ tail /var/log/httpd/error_log
```

What was the effect of HUP on Apache?

16.While running X, start some interactive processes in the background and experiment with using jobs, bg, and fg. For example:

```
$ netscape &  
$ xterm &  
$ emacs &  
$ jobs  
$ fg 1  
$ fg 2  
...
```

Were you able to bring each of the jobs to the foreground successfully?

17. This exercise starts a process, using various methods to view and modify the process execution priority:

1. Start an editing session in the background using nice:

```
$ nice vi &
```

2.Observe that the process was nice'd using ps:

```
$ ps -u
```

3.Check it again using top:

```
$ top -i
```

4.Within top, renice the vi process using the r command and observe the effect on priority.

5.Exit top and use renice to set the nice value back to zero.

18.Use a simple regular expression with grep to find sh and bash users in /etc/passwd.

19.Determine the number of empty lines in /etc/inittab.

20. Use vi to create a text file. Enter insert mode with i and insert text. Quit insert mode with Esc and move around using h, j, k, and l, then re-enter insert mode and add more text. End the session with ZZ. cat the file. Is it as expected?

Devices, Linux Filesystems, and the Filesystem Hierarchy Standard

1. What are the three types of disk partitions found on a Linux system? Which type can contain other partitions and which type does it contain?
2. Name the directories that must be within the root partition.
3. Describe the differences between physical disks, partitions, and filesystems.
4. What is a swap partition used for? Why not just use swap files?
5. What kind of output will df -h yield?
6. Describe a common situation that is likely to cause the automatic use of fsck on the next system boot.
7. Name the fields in /etc/fstab.
8. Give the command to mount a CD-ROM drive on the secondary master IDE device, assuming that /etc/fstab does not contain a line for the device.
9. If the ro option is used in /etc/fstab for /usr, what limitation is placed on that filesystem?
10. Compare and contrast hard and soft quota limits.
11. What three types of users can be granted or denied access to filesystem objects and how do they apply to files and directories?
12. Name the symbolic permission that is equivalent to 0754.
13. Describe a situation that requires the SUID permission. What ramifications does this permission imply?
14. Compare and contrast the differences between hard and symbolic links.
15. Name the document to which Linux directory assignments should conform.
16. Compare and contrast the differences between the locate and find commands.

Exercises

1. As root, run fdisk and enter the p command to print the partition table. Examine your system's configuration and make sure you understand everything you see. Enter the l command and review the many partition types Linux can accommodate. Enter the q command to quit without saving changes.
2. If you have available disk space, use fdisk to create a new ext2 partition, then format it with mkfs. Pay close attention to the output from mkfs.
3. Use a pager to examine /var/log/messages and search for entries made by fsck. Did it find any problems?
4. If you created a new partition in the previous exercises, check it with fsck and observe the output:

```
$ fsck -f /dev/ partition
```

5. Check on the status of filesystems using df:

```
$ df -h
```

1. How does the -h flag assist you with interpreting the results?
2. Are any of your filesystems nearly full?
3. Which are underutilized?

6. As root, get a top-level view of disk usage by user using du:

```
$ du -s /home/*
```

1. Are there any surprises?
7. How could you use sort to make this output more useful?
8. Review /etc/fstab. Be sure you can name all six fields and their order as well as describe their function.
9. Examine the output of the mount command without options. Compare the output with the contents of /etc/fstab.
10. If you created a new partition in the previous exercises, mount it on /mnt/new or some other location of your choosing:

```
$ mkdir /mnt/new  
$ mount /dev/ partition /mnt/new  
$ df /mnt/new
```

1. Did the filesystem mount correctly? Can you store files on it?
2. Next, unmount it:

```
$ umount /dev/ partition /mnt/new
```

3. Add a line to /etc/fstab for the new partition:

```
/dev/partition /mnt/new ext2 defaults 1 2
```

11. Test the quotas by setting them low for a particular user, then start adding files as that user until the quota is exceeded. What is the observable consequence of exceeding the quota?**12. Practice converting these file modes from octal to symbolic form:**

1. 0777
2. 0754
3. 0666
4. 1700
5. 7777

13. Practice converting these file modes from symbolic to octal form. You can assume that x bits are set under SUID, SGID, and sticky bits:

1. -rwxr-xr-x
2. -r--r--r--
3. -rwsrwsrwx
4. -rw-rw---t
5. -rws-w--w-

14. Create temporary files and use chmod with both symbolic and numeric mode modifications. Include SUID, SGID, and sticky bits.

15. As root, create temporary files and use chown to modify user ownership and group ownership.

16. Use chgrp to modify group ownership on the temporary files created in the previous exercise.

17. Create a temporary file and links as follows:

```
$ touch a_file
$ ln -s a_file an_slink
$ ln a_file an_hlink
```

18. Now verify that the file and the hard link indeed share an inode and that the symbolic link points to the original file:

```
$ ls -li a_file an_slink an_hlink
```

19. Read the latest version of the FHS (it's not very long).

20. Examine your filesystem. Does it match the FHS? If you find discrepancies, is it clear why they don't?

21. Use which to check on the location of executable files.

22. Use find to search for bash:

```
$ find / -name bash
```

Now use locate for the same file:

```
$ locate bash
```

How are the results different? Describe a context in which each command would be useful.

23. Update your locate database using updatedb. Note the amount of time this command takes and the resources it consumes on your system.

The X Window System

1. When using XFree86 v3.3.x, what software installation may be required when changing to a different video chipset, and why?

2. Describe how the location of fonts is conveyed to the XFree86 X server.

3. How is the use of a font server different from the use of a font path?
4. Which file controls access to xdm by remote X terminals?
5. Describe the function of xinit.
6. Compare and contrast a window manager, a desktop environment, and an X server.
7. Name the three components of the DISPLAY environment variable.

Exercises

1. From <http://www.xfree86.org/>, obtain XFree86 in the precompiled binary form for your system using the instructions found in Chapter 7. Start with the Xinstall.sh script and run it with `./Xinstall.sh -check` to determine which package to get.
2. Back up your old installation (`/etc/X11` and `/usr/X11R6`). Install the new version using Xinstall.sh as directed in the instructions accompanying the package.
3. Use `xf86config` to configure the new X server. Are you able to get an X display? Is the resolution correct?
4. Try generating the X configuration using `XF86Setup` or `xf86cfg` (depending on the X version). Is the program successful? Does it yield a working X configuration?
5. Obtain a new Type 1 font (or TrueType font, if you are using XFree86 4.x) from the Internet (try a search at <http://www.google.com> or your favorite search engine). Add the font to `/usr/X11R6/lib/fonts/local` and use the `mkfontdir` utility on that directory. Add the directory to your font path (with `xset fp+ /usr/X11R6/lib/fonts/local`). Use `xfontsel` to view the new font. Was the font added correctly?
6. Configure `xfs` as described in Chapter 7, and remove the `FontPath` statements from `XF86Config`, substituting `unix/-1`. Start the font server and restart the X server. Using `xfontsel`, are you able to verify the availability of the same fonts as were available before?
7. Start `xdm` on your system, and note that the system starts the X server and presents a login prompt. If all works correctly, change the default runlevel to that which starts `xdm` and reboot. Does the system now present a graphical login screen?
8. Examine the scripts and programs used on your system for starting X, beginning with `startx`. Look for how the system uses files in the user's home directory to control GUI startup.
9. Examine your window manager configuration. Which window manager are you using? Is it part of a desktop environment, such as KDE or GNOME? Determine where its menus are configured and add an item or two to the menus.